

1. What is Big Data?

Big Data refers to extremely large and complex datasets that cannot be stored, processed, or analyzed efficiently using traditional database systems and tools. These datasets are generated from various sources such as social media, online transactions, mobile applications, sensors, IoT devices, healthcare systems, and business operations.

Big Data is characterized not only by its large size but also by the speed at which it is generated and the variety of formats in which it exists. Organizations use Big Data technologies to collect, store, process, and analyze this information to gain valuable insights, improve decision-making, predict trends, and optimize business processes.

Examples:

- Facebook posts, comments, and likes
- YouTube videos and user activity
- Online shopping transactions
- Banking and financial records
- Sensor data from smart devices
- Healthcare and medical records

Characteristics of Big Data :

1. **Volume** – Massive amounts of data generated every day.
2. **Velocity** – Data is generated and processed at very high speed.
3. **Variety** – Data comes in different formats such as text, images, videos, audio, and logs.
4. **Veracity** – Refers to the quality, accuracy, and reliability of data.
5. **Value** – The useful insights and benefits obtained from analyzing the data.

2. What is Pyspark?

PySpark is the Python API for Apache Spark, a powerful open-source framework used for processing and analyzing large amounts of data quickly and efficiently. It allows

developers and data engineers to write Spark applications using the Python programming language while taking advantage of Spark's distributed computing capabilities.

Traditional Python programs process data on a single machine, which can become slow when dealing with huge datasets. PySpark distributes the data and processing tasks across multiple machines, making data processing much faster and more scalable.

PySpark = Spark + Python

Features of PySpark

- Processes large volumes of data efficiently.
- Supports distributed computing.
- Provides DataFrame and SQL functionalities.
- Integrates with Hadoop, Hive, and cloud platforms.
- Supports machine learning through MLlib.
- Handles both batch and real-time data processing.

Advantages of PySpark

1. **High Performance** – Processes data much faster than traditional methods.
2. **Scalability** – Can handle terabytes and petabytes of data.
3. **Easy to Learn** – Uses Python syntax, which is simple and popular.
4. **Fault Tolerance** – Automatically recovers from failures.
5. **Supports Multiple Data Sources** – Works with CSV, JSON, Parquet, Hive, databases, and more.

Example:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("MyFirstApp") \
    .getOrCreate()
data = [(1, "Rahul"), (2, "Priya")]
```

```
df = spark.createDataFrame(data, ["id", "name"])
df.show()
```

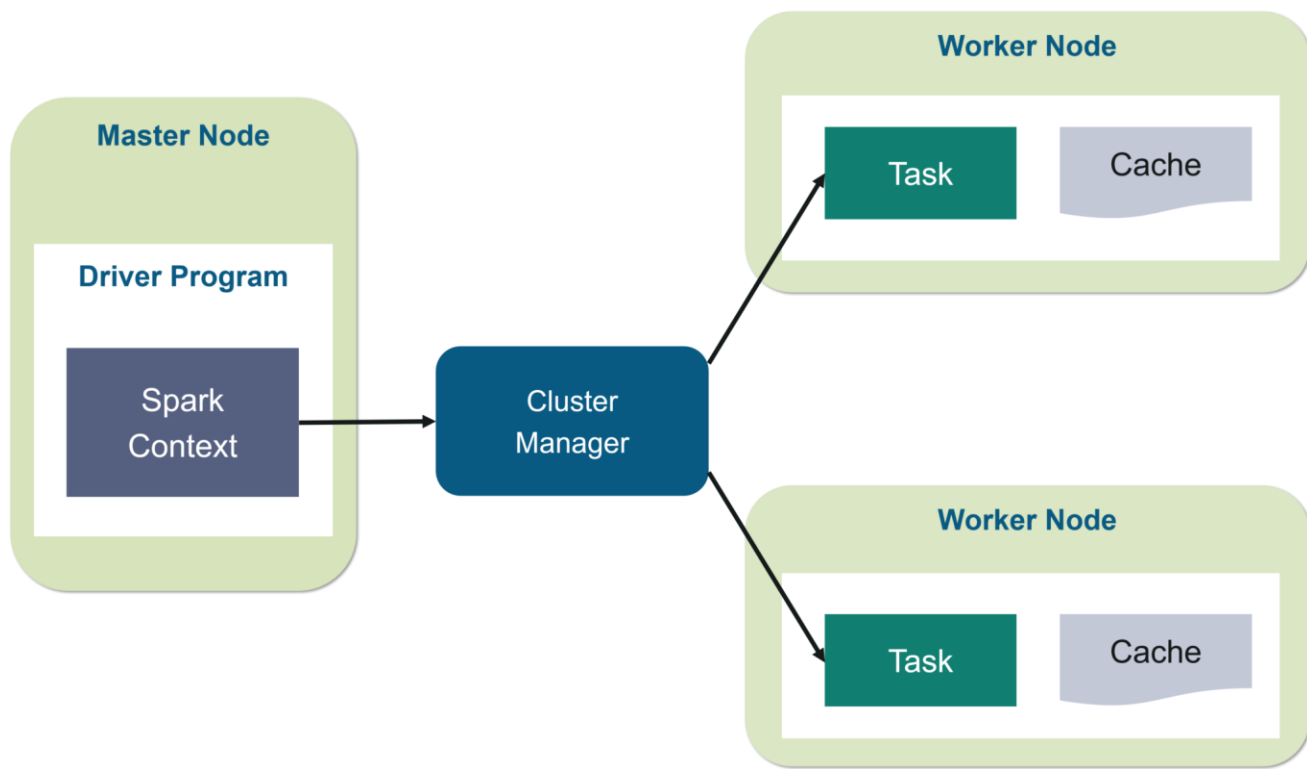
Output

id	name
1	Rahul
2	Priya

3. Spark Basic Architecture

1. Master Node

The Master Node is the main node in the Spark cluster that controls and manages the entire Spark application. It contains the Driver Program and coordinates the execution of tasks across all Worker Nodes.



2. Driver Program

The Driver Program is the brain of the Spark application. It runs the user's code, creates the execution plan, divides jobs into tasks, and collects results from Worker Nodes.

3. Spark Context

The Spark Context is the core component that connects the Spark application to the cluster. It communicates with the Cluster Manager, requests resources, and distributes tasks to Worker Nodes.

4. Cluster Manager

The Cluster Manager is responsible for managing the resources of the cluster. It allocates CPU, memory, and Worker Nodes for Spark applications and ensures efficient utilization of resources.

5. Worker Node

A Worker Node is a machine in the cluster that performs the actual data processing. It receives tasks from the Driver Program and executes them using available resources.

6. Task

A Task is the smallest unit of work in Spark. Large jobs are divided into multiple tasks, which are executed in parallel across different Worker Nodes to improve performance.

7. Cache

Cache is a memory storage area within Worker Nodes where frequently used data is stored. Caching helps Spark avoid repeated disk reads, significantly improving the speed of data processing and analysis.

3.1. Cluster Manager

The Cluster Manager is responsible for managing and allocating resources in the Spark cluster. It assigns CPU, memory, and Worker Nodes to Spark applications and ensures that tasks are distributed efficiently across the cluster.

3.2. Spark Session

The Spark Session is the entry point to PySpark. It allows users to create DataFrames, read and write data, execute SQL queries, and access all Spark functionalities. Every PySpark application starts by creating a Spark Session.

3.3. Spark Context

The Spark Context is the core engine of Spark that connects the application to the cluster. It communicates with the Cluster Manager, requests resources, and distributes jobs and tasks to Worker Nodes for execution.

3.4. Driver Node

The Driver Node is the master node that runs the Driver Program. It acts as the brain of the Spark application by creating the execution plan, dividing jobs into tasks, sending tasks to Worker Nodes, and collecting the final results.

3.5. Worker Node

A Worker Node is a machine in the Spark cluster that performs the actual data processing. It receives tasks from the Driver Node, executes them using executors, and returns the results after processing the assigned data.

4. What is client mode and cluster mode?

Client Mode

In Client Mode, the Driver Node runs on the machine from which the Spark application is submitted (such as your laptop or local system). The Driver communicates with the Cluster Manager and Worker Nodes to execute tasks. This mode is mainly used for development, testing, and debugging because the logs and output are easily accessible on the local machine. However, if the client machine disconnects or shuts down, the Spark application may stop.

Cluster Mode

In Cluster Mode, the Driver Node runs inside the cluster on one of the cluster machines instead of the user's local system. After submitting the application, the cluster takes full responsibility for executing it. This mode is more reliable and is commonly used in production environments because the application continues running even if the user's machine disconnects.

Difference

Feature	Client Mode	Cluster Mode
Driver Location	Local Machine	Inside Cluster
Debugging	Easy	Slightly Difficult
Reliability	Lower	Higher
Client Disconnect	Job may stop	Job continues
Usage	Development & Testing	Production